

《面向对象程序设计》之——

文件和流

连 盛

shenglian@fzu.edu.cn

计算机与大数据学院

福州大学

本章重点

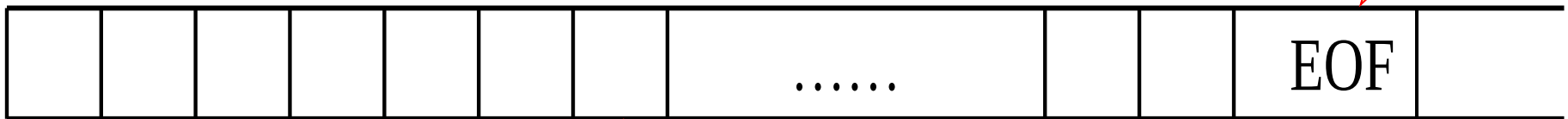
- 文件和流的概念
 - 使用FILE结构操作文件
 - 使用特殊的类和对象操作文件
- ❖ 文件对于实际系统的开发非常有用，而且不可替代

基本概念 (1/3)

■ 文件是什么？

- 文件是外存数据的组织单位。一个文件对应一批存放在外存（如硬盘、软盘、U盘）的数据，计算机以及计算机的使用者通过文件名实现对文件的操作（如复制、删除、内容修改）。
- C++把一个文件看成是一个字节序列。

0 1 2 3 4 5 6 n-2 n-1



结束符

字节中的内容？

基本概念 (2/3)

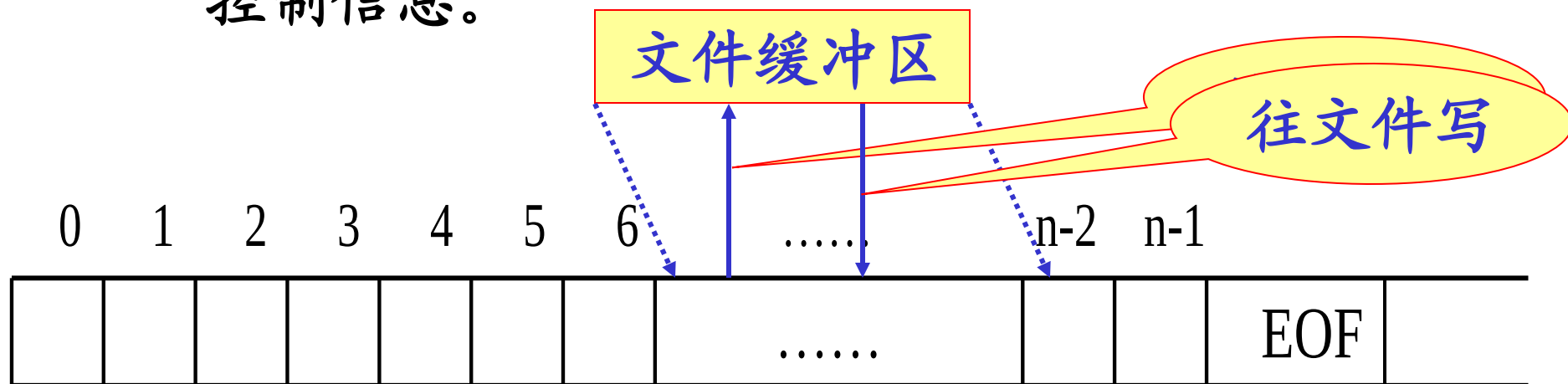
■ C++的文件类型

- 文本文件（也称ASCII文件）。文件中每个字节的内容是ASCII码（0~127），便于阅读和编辑。文件名通常以.txt为后缀
- 二进制文件。文件中每个字节的内容不限（八个二进制位的任意0/1组合），无法阅读和编辑。文件名通常以.dat为后缀
- ❖ 在目前的实际C++系统中，对上述两种类型的文件都不限制字节内容。两者仍有差别

基本概念 (3/3)

■ 流是什么？

- 流是对文件中字节流向及顺序的描述。一个流对应一个正在工作的文件（指文件被打开后，被关闭前），实质上是计算机操作系统为文件建立的一个保存文件部分内容的缓冲区及有关控制信息。



用FILE结构操作文件 (1/9)

■ 三部曲

- 打开一个已存在的文件或建立并打开一个新文件
 - 关键函数: `fopen`
- 对该文件进行读/写操作
 - 关键函数: `fgetc`、`fputc`、`fgets`、`fputs`、`fscanf`、`fprintf`、`feof`、`rewind`、`fseek`、`fread`、`fwrite`
- 关闭该文件
 - 关键函数: `fclose`
- ❖ 以上函数都在库文件`stdio.h`中定义

用FILE结构操作文件 (2/9)

■ 三部曲之一

■ 函数原型:

```
FILE *fopen( const char *filename,  
             const char *mode );
```

filename—路径及文件名, 如

“c:\\x\\xs\\x1.txt”、“x2.txt”、“x3.dat”

mode—打开方式, 如 “r”、“r+”、“rb”、“rb+”

用FILE结构操作文件 (3/9)

■ 三部曲之一（续）

■ FILE * : 文件指针

FILE是在库文件`stdio.h`中定义的一个结构类型。C++用该类型的结构描述一个流，其重要数据有：文件名、操作方式、文件及缓冲区当前位置、文件结束标记

用FILE结构操作文件 (4/9)

- 三部曲之一（续）

- **mode** :

C++有12种不同的文件打开方式，
见下页

文本文件的六种打开方式

方式	描述
r	打开一个供读取数据的文件。
w	建立一个供写入 数据的文件。 如果文件已经 存在， 则废弃 原来的 内容。最后打开文件。
a	建立或打开 一个把数据 追加到文件尾部 的文件。
r+	打开一个 用于更新数据(读和写)的文件。
w+	建立一个用于更新数据的文件。如果文件已经 存在， 则废弃 原来的 内容。最后打开文件。
a+	建立或打开 一个用于更新的文件， 写入的数据追加到文件尾部。

二进制文件的六种打开方式

方式	描述
rb	打开一个供读取数据的文件。
wb	建立一个供写入 数据的文件。 如果文件已经 存在， 则废弃 原来的 内容。最后打开文件。
ab	建立或打开 一个把数据 追加到文件尾部 的文件。
rb+	打开一个 用于更新数据(读和写)的文件。
wb+	建立一个用于更新数据的文件。 如果文件已经 存在， 则废弃 原来的 内容。最后打开文件。
ab+	建立或打开 一个用于更新的文件， 写入的数据追加到文件尾部。

用FILE结构操作文件 (5/9)

■ 三部曲之二：关键函数的原型

- `int fgetc(FILE *);`
- `int fputc(int, FILE *);`
- `char * fgets(char *, int, FILE *);`
- `int fputs(const char *, FILE *);`
- `int fscanf(FILE *,
 const char *, ...);`
- `int fprintf(FILE *,
 const char *, ...);`

function

<stdio.h>

fgetc

```
int fgetc ( FILE * stream );
```

Get character from stream

Returns the character currently pointed by the internal file position indicator of the specified **stream**. The internal file position indicator is then advanced to the next character.

If the stream is at the end-of-file when called, the function returns [EOF](#) and sets the **end-of-file indicator** for the stream ([feof](#)).

If a read error occurs, the function returns [EOF](#) and sets the **error indicator** for the stream ([ferror](#)).

`fgetc` and [getc](#) are equivalent, except that [getc](#) may be implemented as a macro in some libraries.

Parameters

`stream`

Pointer to a [FILE](#) object that identifies an input stream.

Return Value

On success, the character read is returned (promoted to an `int` value).

The return type is `int` to accommodate for the special value [EOF](#), which indicates failure:

If the position indicator was at the **end-of-file**, the function returns [EOF](#) and sets the **eof indicator** ([feof](#)) of **stream**.

If some other reading error happens, the function also returns [EOF](#), but sets its **error indicator** ([ferror](#)) instead.

<https://cplusplus.com/reference/cstdio/fgetc/>

用FILE结构操作文件 (7/9)

■ 三部曲之二：关键函数举例

fgetc: 从文件中读取一个字符

- 参数: 文件指针

- `fgetc( stdin )` 相当于 `getchar()`

fputc: 向文件中写入一个字符

- 参数: 拟写入的字符和文件指针

- `fputc( 'a' , stdout )` 相当于 `putchar( 'a' )`

用FILE结构操作文件 (8/9)

fseek: 修改文件位置指针

```
int fseek(FILE *, long, int);
```

第一个参数——文件指针

第二个参数——偏移量

第三个参数——基准，可以为SEEK_SET、SEEK_CUR和SEEK_END，分别表示文件的起始位置、当前位置和文件尾

用FILE结构操作文件 (9/9)

- 三部曲之三

- 函数原型:

- ```
int fclose(FILE *);
```





例子1: 使用FILE指针新建一个文本文件,  
写入了字符串 "This is a test file. "



```
#include <stdio.h>
main()
{FILE *fptr; //定义FILE指针
 //打开或创建文件
 if ((fptr = fopen("test.txt", "w"))
 != NULL)
 { //往文件中写入字符串
 fprintf(fptr, "This is a test file.");
 fclose(fptr); //关闭文件
 }
 else printf("Open file error.\n");
 return 0;
}
```

## 例子2: 将例1创建的文件中的数据读出来



```
#include <stdio.h>
main()
{FILE *fptr; //定义FILE指针
 int c;
 if ((fptr = fopen("test.txt", "r"))
 != NULL)
 {c=fgetc(fptr);
 while(!feof(fptr)) //注意feof的用法
 {putchar(c);
 c=fgetc(fptr);
 }
 fclose(fptr); //关闭文件
 }
 else printf("Open file error.\n");
 return 0;
}
```

# fread

```
size_t fread (void * ptr, size_t size, size_t count, FILE * stream);
```

## Read block of data from stream

Reads an array of **count** elements, each one with a size of **size** bytes, from the **stream** and stores them in the block of memory specified by **ptr**.

The position indicator of the stream is advanced by the total amount of bytes read.

The total amount of bytes read if successful is (size\*count).

## Parameters

ptr

Pointer to a block of memory with a size of at least (size\*count) bytes, converted to a void\*.

size

Size, in bytes, of each element to be read.

[size\\_t](#) is an unsigned integral type.

count

Number of elements, each one with a size of **size** bytes.

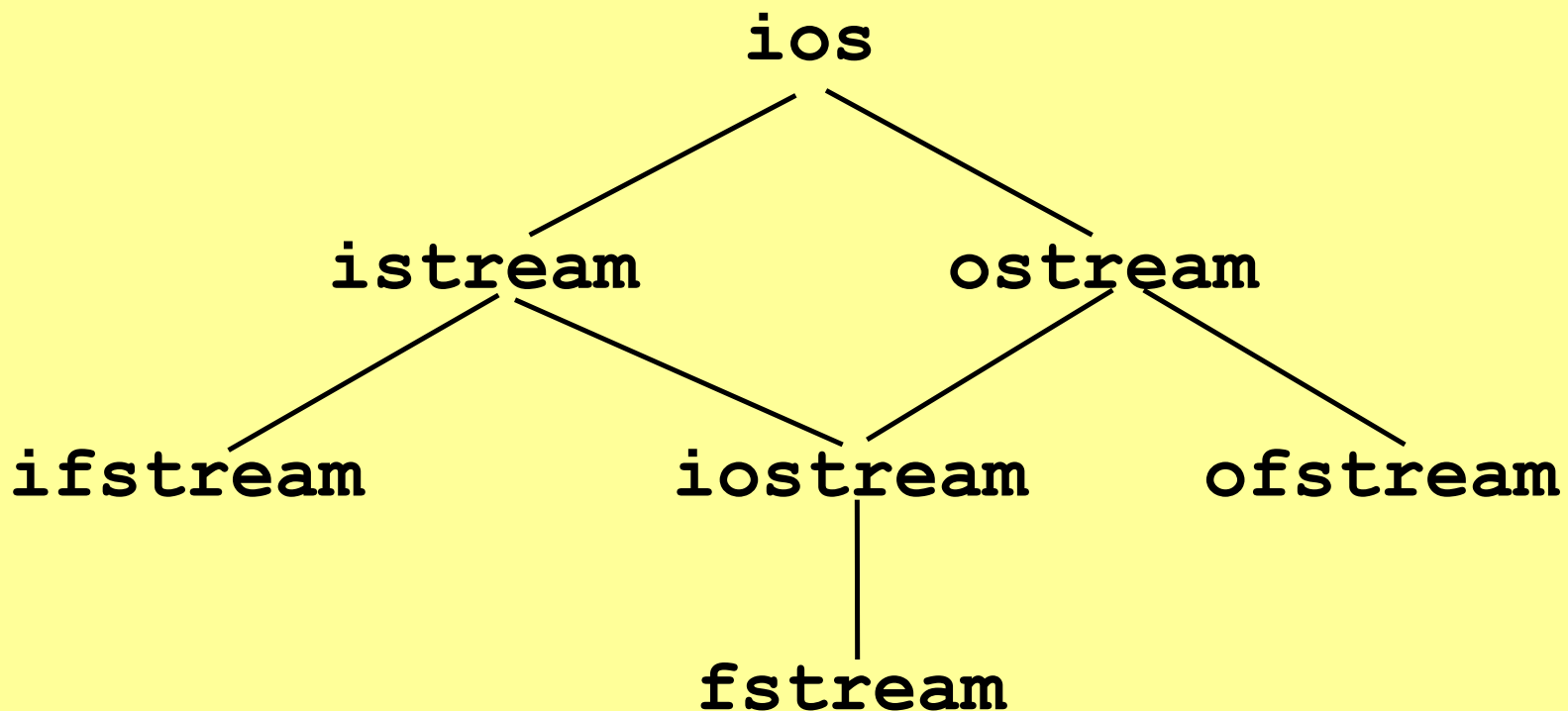
[size\\_t](#) is an unsigned integral type.

stream

Pointer to a [FILE](#) object that specifies an input stream.

# 使用类和对象操作文件 (1/6)

## ■ 文件流类的继承关系



# C++的I/O对C的发展

## ■ 类型安全

- `printf ( “%d” , f );`
- `printf ( “%d” , “C++” );`
- `scanf ( “%d” , i ) ;` (漏写`&`?)

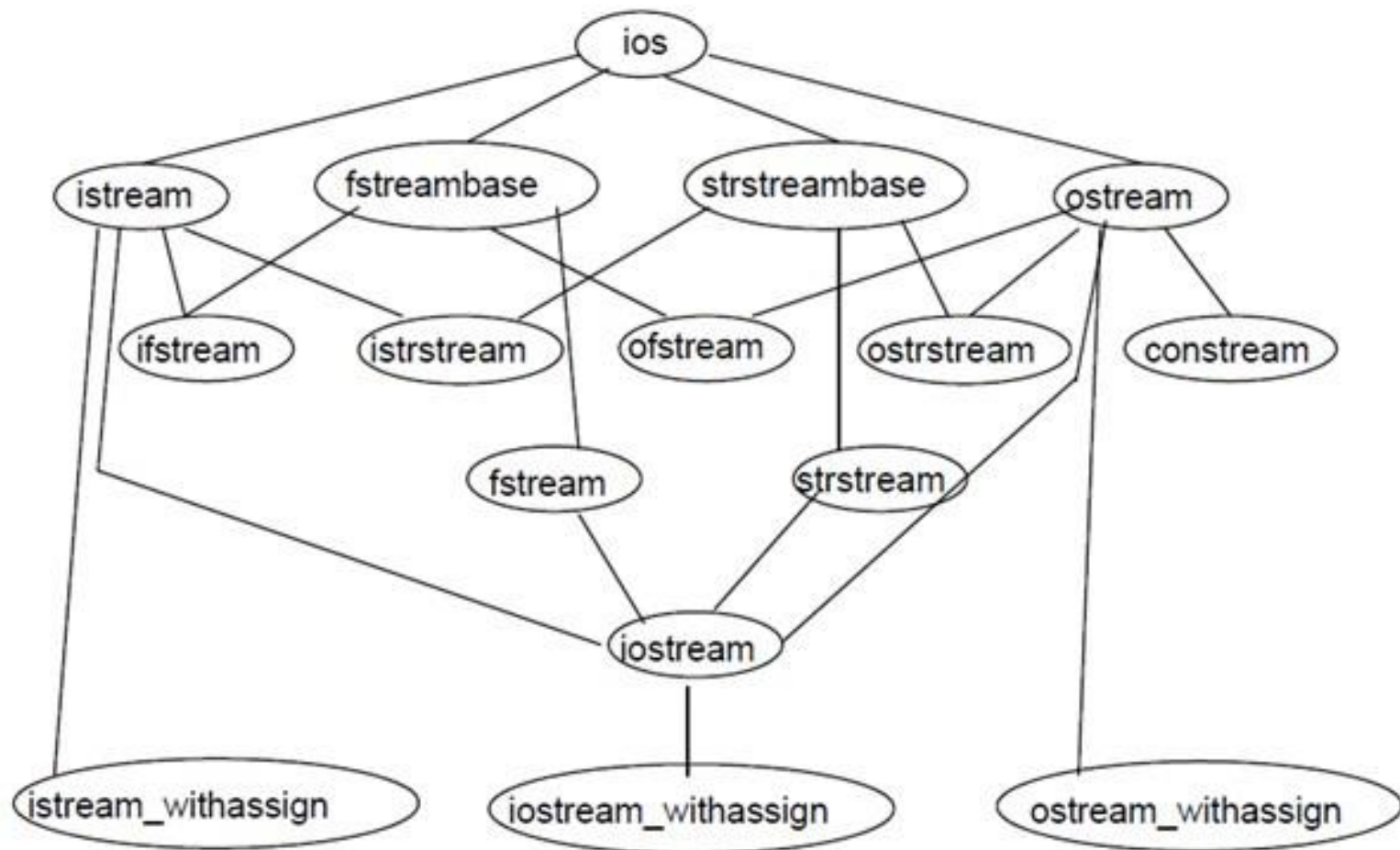
## ■ 可扩展性

- `printf` 和 `scanf` 只能输入输出标准类型的数据



# 使用类和对象操作文件 (1/6)

## ■ 文件流类的继承关系



Reference

▶ C library:

▶ Containers:

▼ Input/Output:

<fstream>

<iomanip>

<ios>

<iosfwd>

<iostream>

<istream>

<ostream>

<sstream>

<streambuf>

▶ Multi-threading:

▶ Other:

<iostream>

▼ objects

cerr
cin
clog
cout
wcerr
wcin
wclog

header

# <iostream>

## Standard Input / Output Streams Library

Header that defines the standard input/output stream objects:

C++98

C++11

?

Including this header may automatically include other headers, such as [<ios>](#), [<streambuf>](#), [<istream>](#), [<ostream>](#) and/or [<iosfwd>](#).

Note that the [iostream](#) class is mainly declared in header [<istream>](#).

### Objects

Narrow characters (char)

|                      |                                             |
|----------------------|---------------------------------------------|
| <a href="#">cin</a>  | Standard input stream (object)              |
| <a href="#">cout</a> | Standard output stream (object)             |
| <a href="#">cerr</a> | Standard output stream for errors (object)  |
| <a href="#">clog</a> | Standard output stream for logging (object) |

Wide characters (wchar\_t)

# 使用类和对象操作文件 (2/6)

- 几个特殊的流对象
  - 标准输入流对象 `cin`  
(类 `istream` 的对象)
  - 标准输出流对象 `cout`
  - 错误流对象 `cerr`, `clog`

# 重载输入(提取)和输出(插入)运算符

**cin>>a;**

**cout<<a;**

对象不能直接输入输出

```
class A
{ float x, y;
public:
 A(float a=0, float b=0){ x=a; y=b; }
 void Set(float a, float b){ x=a; y=b; }
 void Show(void){ cout<<x<<'\\t'<<y<<endl; }
};

void main(void)
{ A a(2,3);
 a.Set(20, 30);
 a.Show();
}
```

输入对象数据

输出对象数据

在C++中允许用户重载运算符“<<”和“>>”，  
实现对象的输入和输出。重载这二个运算符时，在  
对象所在的类中，将重载这二个运算符的函数说  
明该类的友元函数。

重载提取运算符的一般格式为：

返回值类型      函数名      左操作数      右操作数

friend istream & operator >> (istream &, ClassName &);

友元函数

cin>>a;

operator>>(cin, a)

返回值类型

函数名

左操作数

右操作数

```
friend istream & operator >> (istream &, ClassName &);
```

友元函数

```
istream & operator >> (istream &is, ClassName &f);
```

```
cin >> a; operator >> (cin, a)
```

返回值类型：类 `istream` 的引用，`cin` 中可以连续使用运算符 “>>”。

```
cin >> a >> b;
```

第一个参数：是 “>>” 的左操作数 `cin` 类型，类 `istream` 的引用

第二个参数：是 “>>” 的右操作数，即欲输入的对象引用。

```
class A
{ float x, y;
 public:

 friend istream & operater >>(istream &, A &);
};
```

在类中原型说明

```
.....
friend istream & operater >>(istream &, A &);
```

在类外定义函数

```
.....
A a;
cin>>a;
....
```

```
istream & operater >>(istream &is, A &a)
{ cout<<" Input a:"<<endl;
 is>>a.x>>a.y;
 return is;
}
```

重新定义输入流

返回输入流

```
class incount{
 int c1,c2;
public:incount(int a=0,int b=0) { c1=a; c2=b; }
 void show(void){cout<<"c1="<<c1<<"\t"<<"c2="<<c2<<endl;}
 friend istream & operator>>(istream &,incount &);
};
```

重载输入函数原型说明

```
istream & operator>>(istream &is, incount &cc)
{ is>>cc.c1>>cc.c2; return is; }
```

```
void main(void)
{
 incount x1,x2;
 x1.show ();
 x2.show ();
 cin>>x1;
 cin>>x2;
 x1.show ();
 x2.show ();
}
```

重载输入函数定义



# 重载输出（插入）运算符的一般格式为：

返回值类型

函数名

左操作数

右操作数

friend ostream & operator <<(ostream &, ClassName &);

友元函数

cout<<a; **operator<<**(cout, a)

## 与输入（提取）运算符比较：

friend istream & operator >>(istream &, ClassName &);

将输入流改为输出流。

```
class A
{ float x, y;
 public:

 friend ostream & operator << (ostream &, A &);
};
```

在类中原型说明

```
.....
friend ostream & operator << (ostream &, A &);
```

在类外定义函数

```
.....
A a(2,3);
cout<<a;
....
```

```
ostream & operator <<(ostream &os, A &a)
{ cout<<" The object is:"<<endl;
 os<<a.x<<"\t"<<a.y<<endl;
 return os;
}
```

重新定义输出流

返回输出流

```

class incount{
 int c1,c2;
public: incount(int a=0,int b=0) { c1=a; c2=b; }
 void show(void) {cout<<"c1="<<c1<<"\t"<<"c2="<<c2<<endl; }
 friend istream & operator>>(istream &,incount &);
 friend ostream & operator<<(ostream &,incount &);
};

istream & operator>>(istream &is,incount &cc)
{ is>>cc.c1>>cc.c2; return is;}

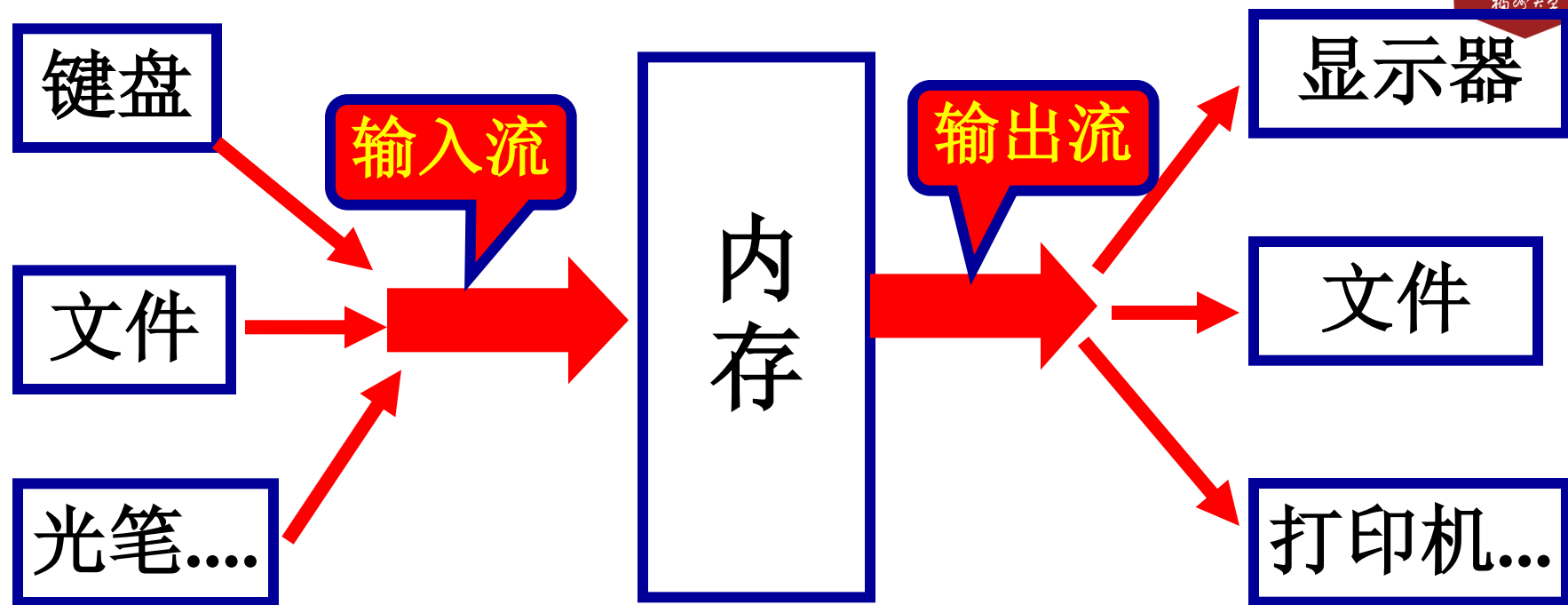
ostream &operator<<(ostream &os,incount &cc) //重载cout<<
{os<<"c1="<<cc.c1<<"\t"<<"c2="<<cc.c2<<endl; return os;}

void main(void)
{
 incount x1,x2;
 cout<<x1<<x2;//调用输出函数
 cin>>x1; //调用输入函数
 cin>>x2;
 cout<<x1<<x2;
}

```

重载输出函数原型说明

重载输出函数定义



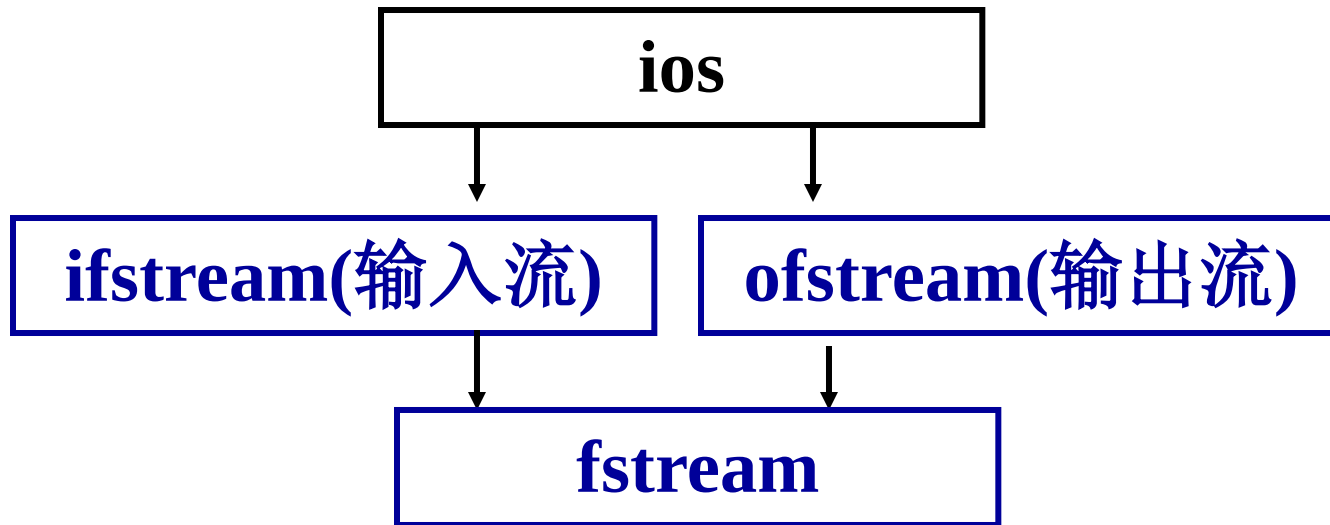
编译系统已经以运算符或函数的形式做好了对标准外设（键盘、屏幕、打印机、文件）的接口，使用时只需按照要求的格式调用即可。

`cin>>x;    cout<<x;`

`cin.get(ch);`

# 文件流

C++在头文件**fstream.h**中定义了C++的文件流类体系,当程序中使用文件时, **要包含头文件fstream**。

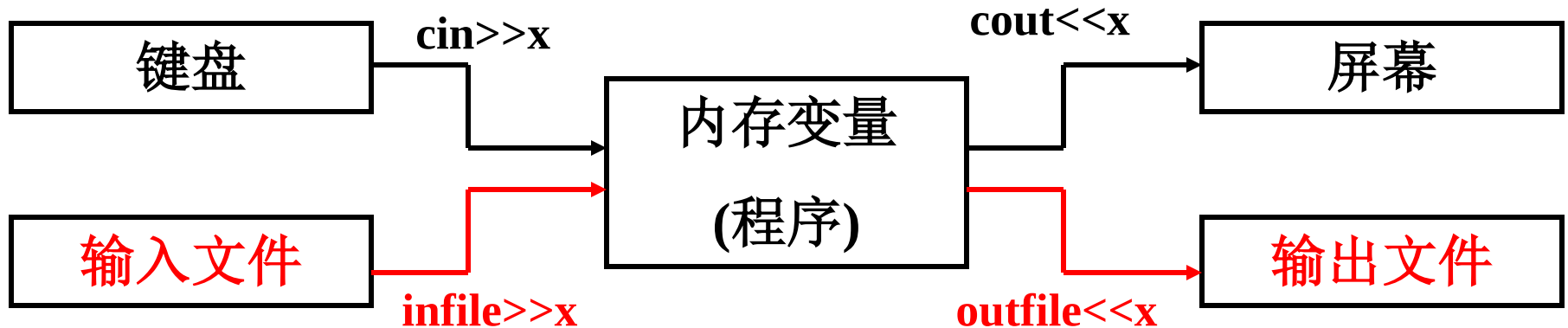


在“**fstream.h**”中说明

当使用文件时, 在程序头有: **#include<fstream>**

其中定义了各种文件操作运算符及函数。

# 程序对文本文件的操作与对键盘、显示器的操作比较：



在涉及**文本文件**的操作时，将输入文件看成键盘，将输出文件看成显示器，格式不变。**只需在程序中增加打开与关闭文件的语句。**

# 文件的操作

文件 { 文本文件：  
以ASCII表示的文件：记事本，\*.cpp等  
二进制文件：  
用二进制形式表示的文件：可执行程序\*.EXE等

56: ASCII表示为 00110101 00110110, 占两字节

56: 二进制表示为 111000, 占六个二进制位

不同的文件操作的函数、格式不同

# 文本文件的打开与关闭

在文件操作前，需要将程序与被操作的文件联系起来，使程序可以“引用”文件。

在程序内定义一个文件类的**对象**，由**该对象**与文件发生联系，程序内所有的与文件的操作都是对**该对象**的操作。

**fstream infile, outfile;**

两个对象，可以联系  
两个输入输出文件

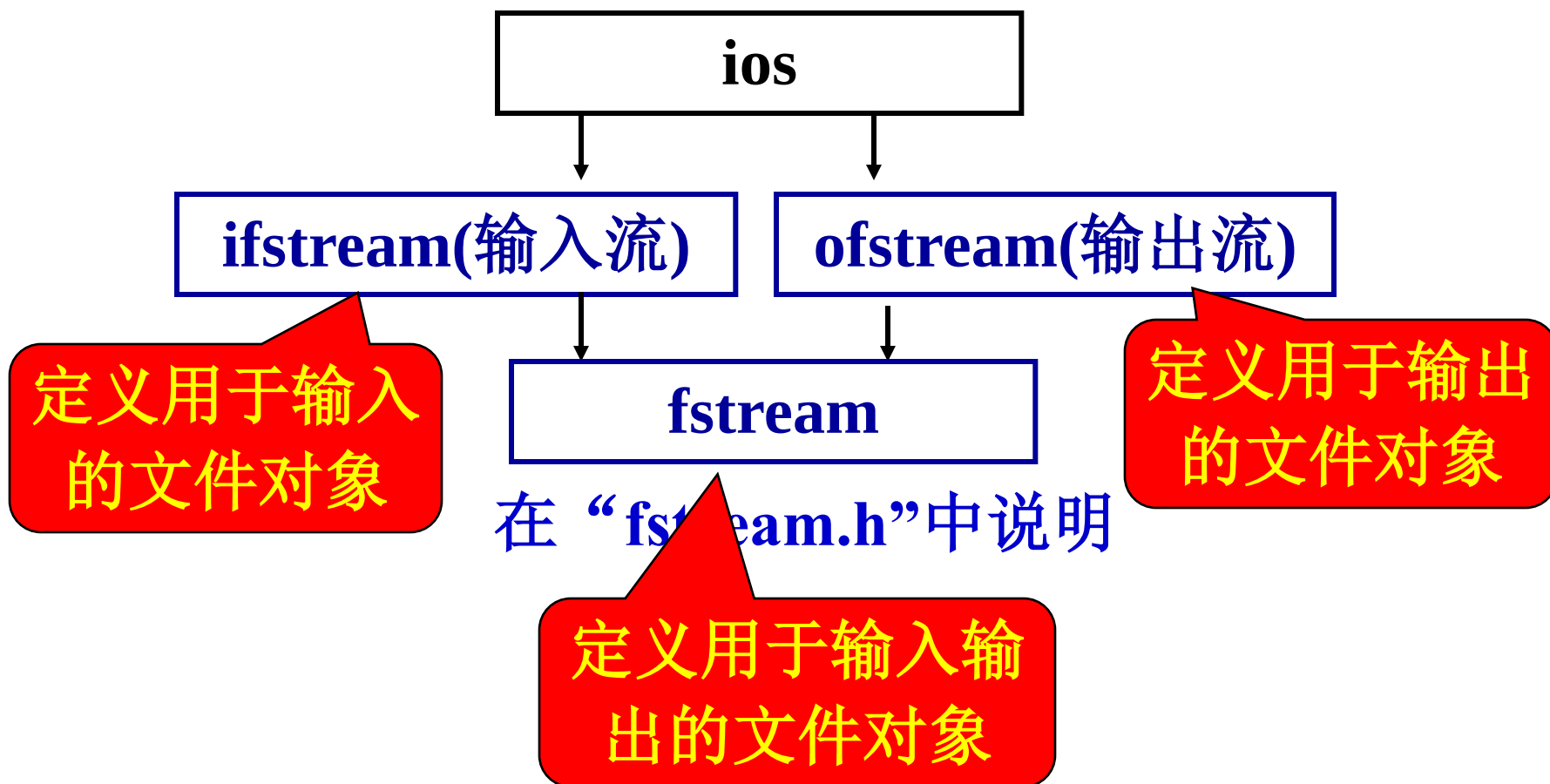
**ifstream infile;**

对象只能联系输入文件

**ofstream outfile;**

对象只能联系输出文件





例子6: 使用文件流对象新建或打开一个文件,  
并写入字符串 "This is a test file. "

```
#include <fstream.h>
main()
{ //声明文件流对象
ofstream outFile("outFile.txt", ios::out);
if (!outFile)
 //使用错误流对象输出错误信息
 cerr<<"Open or create file error."
 <<endl;
else
 //输出数据到与对象outFile 关联的文件中
 outFile << "This is a test file.";
return 0;
}
```

# 如何使文件类的对象与欲操作的文件发生联系？

## 用对象打开文件：

**ifstream infile; //定义输入文件类对象**

**infile.open(“myfile1.txt”);//利用函数打开某一文件**

打开文件的作用是，使文件流对象与要使用的文件名之间建立联系。

**ofstream outfile; //定义输出文件类对象**

**outfile.open(“myfile1.txt”);//打开某一文件供输出**

```
infile.open(“myfile1.txt”);
```

打开文件 “**myfile1.txt**”用于输入，并将这个文件与输入文件类对象**infile**建立联系，今后，在程序中，用到这个文件 “**myfile1.txt**”的地方就用**infile**来代替。

```
outfile.open(“myfile2.txt”);
```

打开文件 “**myfile2.txt**”用于输出，并将这个文件与输出文件类对象**outfile**建立联系，今后，在程序中，用到这个文件 “**myfile2.txt**”的地方就用**outfile**来代替。

# 如何从文件中输入输出数据？

将文件类对象看成键盘和显示器即可。

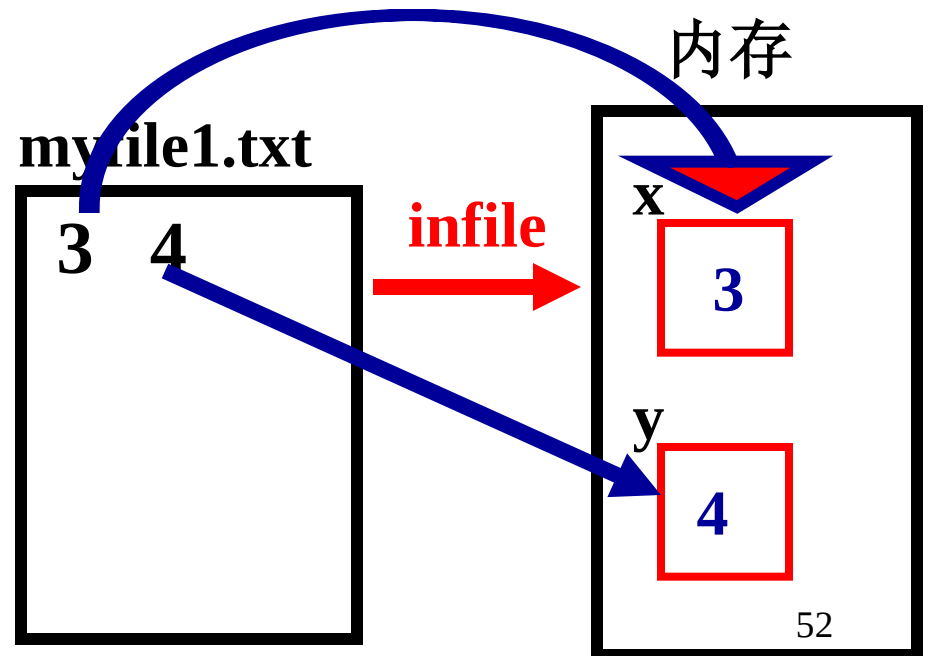
**ifstream infile;** //定义输入文件类对象

**infile.open("myfile1.txt");**//利用函数打开某一文件

**float x , y;**

**infile>>x>>y;**

用 **infile** 代替 **myfile1.txt**  
进行操作。



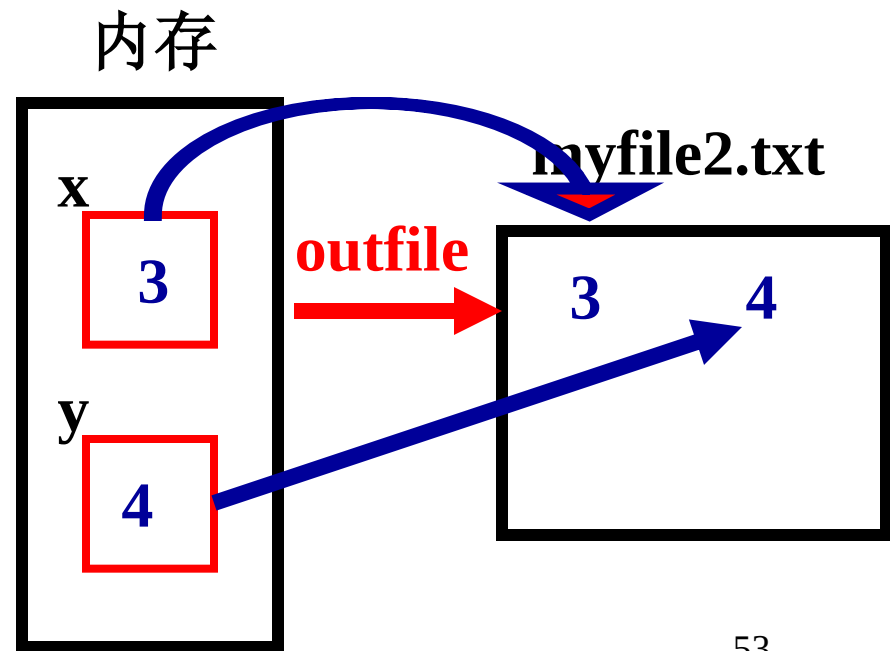
**ifstream outfile;** //定义输出文件类对象

**infile.open("myfile2.txt");**//利用函数打开某一文件

**float x=3 , y=4;**

**outfile<<x<<'\\t'<<y<<endl;**

用 **outfile** 代替 **myfile2.txt**  
进行操作。



4) 用完文件后,使用成员函数关闭文件.如:

`ifstream infile;`

`infile.close();`

`ofstream outfile`

`outfile.close();`

`infile.open("myfile1.txt");`

`outfile.open("myfile2.txt");`

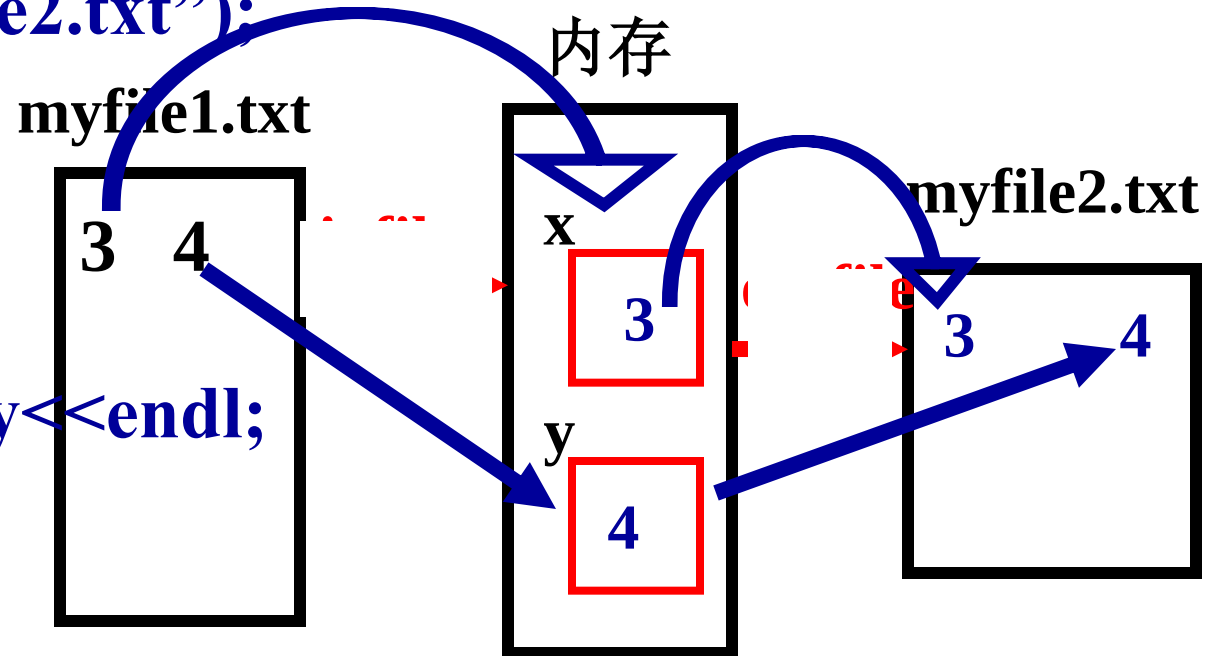
`float x,y;`

`infile>>x>>y;`

`outfile<<x<<'\\t'<<y<<endl;`

`infile.close();`

`outfile.close();`



当用类**fstream**定义文件对象时，该对象即能定义输入文件对象，又能定义输出文件对象，所以打开文件时，必须在成员函数**open()**中的参数中**给出打开方式（读或写）**。

**fstream** pfile1,pfile2;//定义了两个文件类的对象

pfile1.open(“file1.txt”, **ios::in**);//pfile1联系到 “file1.txt”,用于输入

pfile2.open(“file2.txt”, **ios::out**);//pfile2联系到 “file2.txt”,用于输出

char ch;

**pfile1>>ch;** //输入

**pfile2<<ch;** //输出

**pfile1.close();**

**pfile2.close();**



# ● Open a file

The first operation generally performed on an object of one of these classes is to associate it to a real file. This procedure is known as to **open a file**. An open file is represented within a program by a **stream** (i.e., an object of one of these classes; in the previous example, this was `myfile`) and any input or output operation performed on this stream object will be applied to the physical file associated to it.

In order to open a file with a stream object we use its member function `open`:

```
open (filename, mode);
```

Where `filename` is a string representing the name of the file to be opened, and `mode` is an optional parameter with a combination of the following flags:

|                          |                                                                                                                                  |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <code>ios::in</code>     | Open for input operations.                                                                                                       |
| <code>ios::out</code>    | Open for output operations.                                                                                                      |
| <code>ios::binary</code> | Open in binary mode.                                                                                                             |
| <code>ios::ate</code>    | Set the initial position at the end of the file.<br>If this flag is not set, the initial position is the beginning of the file.  |
| <code>ios::app</code>    | All output operations are performed at the end of the file, appending the content to the current content of the file.            |
| <code>ios::trunc</code>  | If the file is opened for output operations and it already existed, its previous content is deleted and replaced by the new one. |

All these flags can be combined using the bitwise operator OR (`|`). For example, if we want to open the file `example.bin` in binary mode to add data we could do it by the following call to member function `open`:



```
1 ofstream myfile;
2 myfile.open ("example.bin", ios::out | ios::app | ios::binary);
```

在打开文件后，都要判断打开是否成功。若打开成功，则文件流对象值为非零值；若打开不成功，则其值为0。

**fstream** pfile1,pfile2;//定义了两个文件类的对象

pfile1.open(“file1.txt”, **ios::in**);

pfile2.open(“file2.txt”, **ios::out**);

if (!pfile1) **若为0，打开文件操作失败**

{cout <<”不能打开输入文件： file1.txt”<<’\n’; exit(1);}

if (!pfile2)

{cout <<”不能打开输出文件： file2.txt”<<’\n’; exit(1);}

# 涉及到字符串的文件读写

`char ch, str[300];`

`ifstream infile("myfile1.txt");`

`ofstream outfile("myfiel2.txt");`

从键盘输入一个字符: `cin.get(ch);`

从文件输入一个字符: `infile.get(ch);`

向显示器输出一个字符: `cout.put(ch);`

向文件输出一个字符: `outfile.put(ch);`

从键盘输入一行字符: `cin.getline(str,300);`

从文件输入一行字符: `infile.getline(ch,300);`

从文件输入一字符或一行字符，当输入至文件尾时，函数返回值为0，可以据此来判断循环结束。

# 实现两文件的拷贝的程序

```
void main(void)
```

```
{ char filename1[256],filename2[256];
```

```
 cout<<"Input source file name: ";
```

```
 cin>>filename1;
```

输入文件(源文件)名

```
 cout<<"Input destination: ";
```

```
 cin>>filename2;
```

输出文件(目的文件)名

```
 ifstream infile(filename1);
```

用构造函数打开文件

```
 ofstream outfile(filename2);
```

```
 char ch;
```

```
 while(infile.get(ch))
```

从源文件中读取一个字符，至文件尾停止循环

```
 outfile.put(ch);
```

```
 infile.close();
```

将该字符输出至目的文件

```
 outfile.close();
```

关闭文件

```
}
```

```
void main(void)
{
 char filename1[256],filename2[256];
 char buf[300];
 cout<<"Input source file name: ";
 cin>>filename1;
 cout<<"Input destination: ";
 cin>>filename2;
 fstream infile,outfile;
 infile.open(filename1,ios::in);
 outfile.open(filename2,ios::out);
 while(infile.getline(buf,300))
 outfile<<buf<<endl;
 outfile.close();
 infile.close();
}
```

输入文件(源文件)名

输出文件(目的文件)名

用函数打开文件

从源文件中读取一行字符，至文件尾停止循环

将该行字符输出至目的文件

关闭文件

# 二进制文件的读写操作

若在文件的打开方式中没有特别说明，打开的文件均为ASCII码文件，若要打开二进制文件，则要特别说明并用特定的读写函数。

```
fstream infile,outfile;
```

```
infile.open("inf1.dat", ios::in|ios::binary);
```

文件名

输入方式打开

二进制文件

```
outfile.open("outf1.dat", ios::out|ios::binary);
```

文件名

输出方式打开

二进制文件

# 判断二进制文件是否读到了文件尾？

## **infile.eof()**

当到达文件结束位置时，该函数返回一个非零值；**否则返回零。**

```
fstream infile;
```

```
infile.open("data1.dat",ios::in|ios::binary);
```

```
if(!infile) 判断打开是否出错
```

```
{ cout<<"Open Error!\n"; exit(1); }
```

```
char str[300];
```

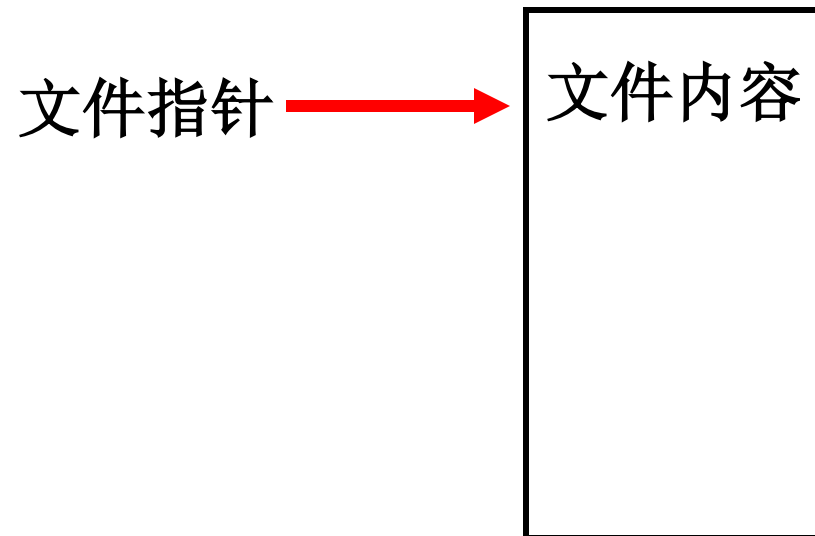
```
while(infile.eof()) 判断是否读到了文件尾
```

```
infile.read(str, 300);
```

# 文件指针

当一打开文件，文件指针位于文件头，并随着读写字节数的多少顺序移动。

可以利用成员函数随机移动文件指针。





# 随机读取二进制文件

**infile.seekg(int);**//将文件指针移动到由参数指定的字节处

**infile.seekg(100);**//将文件指针移动到距离文件头100个字节处

**infile.seekg(int, ios::\_dir);**

移动的字节数

相对位置

**\_dir:** beg: 文件头  
cur: 当前位置  
end: 文件尾

**infile.seekg(100, ios::beg);**//移动到距文件头100个字节

**infile.seekg(-100, ios::cur);**//移动到距当前位置前100个字节

**infile.seekg(-500, ios::end);**//移动到距文件尾前500个字节

# 学习目的检测

- 掌握文件和流的基本概念
- 掌握使用FILE结构操作文本文件和二进制文件的方法
- 了解文件流类的继承关系
- 掌握使用文件流对象操作文本文件和二进制文件的方法

**Thank you !**